

A method may accept any number of parameters, but may return only one data value or object.

As a final example for this section, consider a programmer who is trying to write an object class that represents a book (like this book, only arbitrarily more interesting). Some of the data the programmer may consider are:

- What does a book *have*? A cover, pages, words, ...
- What can a book *do*? It can be read, be skimmed, be put away, ...

From this information, the programmer can begin to write the object class. The programmer must first determine which item(s) are relevant to the situation so as to keep the object class as efficient as possible. For example, if the programmer is designing a book to be used as a paperweight, it is probably not important to discuss how many words are in the book, so that data field would not be incorporated into the program. Likewise, if the programmer is not a student studying a ridiculous number of books in a short amount of time (sound familiar?), then it may not be relevant to have a method that skims the book, rather than reading it closely. The class will then be built based on these decisions.

As you can see, the **planning** of the program is just as important as, if not more important than, the actual writing of the program. In some classes, your teacher may remind you to “write an outline” or “write a first draft” before you compose the final product, and you might skip that step and go right to the end unit and still be successful. For programming at any profound level, which is required in this course to some extent, planning is essential in order for your program to work well and in order to increase your efficiency. Remember, the AP Exam is strictly timed.

The AP Exam tests the planning of a class by presenting a *lengthy* description of the requirements, along with other interacting classes in the project.

COMPOSITION

Now that we have written classes to run a program, we must get these classes to interact correctly in order to create a functioning program. We have the driver, we have the car, now we need to put the driver into the car to eagerly drive to the AP Computer Science A Exam on exam day.



Go Online!

For answers to test-prep questions for all your tests and additional test-taking tips, subscribe to our YouTube channel at youtube.com/ThePrincetonReview



The driver and object classes must reside in the same folder on the computer. If you are using an interactive development environment (as opposed to command line programming), you often have to create a “Project” or some similar structure in the software that will hold all of the program’s files in one place. Remember that the driver depends on the object class in order to compile, and the object class depends on the driver to make it actually do anything.

Consider a program that will take “flower” objects and “plant” them into a garden. The flower objects might be outlined in an object class called `Flower`. One class will perform the planting, so we can call it `MakeGarden`.

Suppose the `Flower` constructor method has the following header:

```
public Flower(int xPos, int yPos)
```

where `xPos` and `yPos` are the horizontal and vertical coordinates in the garden, respectively, where the flower object will be planted. Our driver would have to first “create” the flowers, and then plant them accordingly.

In order to create an object, we must instantiate the object using its corresponding class.

Each time we instantiate a flower object, we must assign it an **object reference variable**; as the fancy name implies, this is just a variable name. Some lines in `MakeGarden` might look like this:

```
Flower f1 = new Flower(2, 1);
Flower f2 = new Flower(2, 2);
Flower f3 = new Flower(2, 3);
```

These lines will instantiate three different flower objects, each containing its own instance data. `f1`, `f2`, and `f3` are used to distinguish between the objects. We can then use these objects to perform methods. Consider a `plant` method in the `Flower` class; this method would “plant” the flower and might have the following header:

```
public void plant()
```

In `MakeGarden`, we must invoke this method through the objects. Let’s say we want to plant only `f1` and `f3`, but not `f2`. We would add the following lines:

```
f1.plant();
f3.plant();
```

Now let’s say the `plant` method, instead, returns whether or not the plant was successfully planted. This altered `plant` method might have the header:

```
public boolean plant()
```

Now we can use the returned data to output an appropriate message in the driver:

```
if (f1.plant())
    System.out.print("Planted successfully.");
else
    System.out.print("There was a problem.");
```

Because we are invoking `f1.plant`, `f1` should be planted as requested. Since the `plant` method returns a boolean value, we can place `f1.plant` in the context of an `if` statement (it produces a truth value so it is considered a condition), and now it also functions as part of a control structure. Awesome!

Let's add to our garden situation. As you know, other items can appear in a garden besides flowers. Plants, bushes, and weeds can appear in gardens. Each of these items would probably have its own object class, since their instance data and methods would be different from those of flowers. For example, a flower has flower petals, while a plant does not; weeds often grow on their own without water, whereas most flowers and plants do not. Regardless, separate object classes can be created for each of these object types, and the driver can hold them all together, as long as they are all stored in the same folder on the computer.

Assuming each of these object classes has been created and each corresponding object class has a `plant` method as outlined above, `MakeGarden` might include these lines:

```
Flower f1 = new Flower();
Flower f2 = new Flower();
Plant p1 = new Plant();
Plant p2 = new Plant();
Weed w1 = new Weed();
if (f1.plant())
    System.out.println("Planted successfully");
if (f2.plant())
    System.out.println("Planted successfully");
if (p1.plant())
    System.out.println("Planted successfully");
if (w1.plant())
    System.out.println("You have a weed.");
```

Note that `MakeGarden` does not instantiate any `Bush` objects, and does not attempt to plant the `p2` object. Note also that we cannot see the entire class! The programmer has the task of deciding which objects to instantiate of which type, when and whether to use them, and how to use them appropriately, based on their class specifications. Still think you can go directly to the final draft?

Let's add another layer. Programmers often recognize that an object is actually composed of many smaller, or more basic, objects. If you think about a flower, it actually has several parts, each of which *has* data and *does* particular actions. As a result, we could write object classes for, say, the stalk and the root system. The `Stalk` class and the `RootSystem` class, then, would reside as attributes of each

flower. Their classes, again, must be placed in the same folder as the rest of the program's classes. The programmer can then set up the `Flower` class as follows:

```
public class Flower
{
    // other data not shown
    private Stalk st;
    private RootSystem rootSys;

    public Flower()
    {
        st = new Stalk();
        rootSys = new RootSystem();
        // other commands not shown
    }

    // other methods not shown
}
```

This means that every `Flower` object *has* a stalk and a root system. The `Flower` class, then, is called an **aggregate class** because it is made up of, among other data, instances of other classes. This setup is more realistic; think of any object in your room, and you can probably see pretty quickly that it is made of smaller objects, each with its own data and methods. A laptop computer has keys, ports, and a screen, which can all be considered objects because they *have* stuff and *do* stuff. A dresser has drawers, but we can go even deeper...a drawer has walls and a floor and can be opened or closed...the floor of a drawer has a shape and is made of a certain material and can be broken if the drawer is overstuffed...you get the idea. Luckily, as programmers, we get to decide the detail of the object classes, and the answer typically lies in the desired function of the program, as we stated before. Do you *really* need to know what material the floor of the drawer is made of? Most people do not, although a furniture retailer might. Again, it all depends on the application.

The free-response questions on the AP Exam will present some sort of context and will often tell you exactly what class(es) and/or method(s) to write, so they don't get a wild variety of solutions from test takers around the world.

Otherwise, they could literally be forced to read thousands of completely different approaches to a given task. This fact is not good for our creativity, but it's great for scoring a 5; save your creative juices for those extra college classes you can take when you AP out of CS!

Use class `Chair` and method `sitOnChair` to answer Questions 3 and 4.

```
public class Chair
{
    private int numberOfLegs = 4;
    private boolean padded;

    public Chair(boolean soft)
    {
        if (soft) padded = true;
        else padded = false;
    }

    public void sitOnChair( )
    {
        <implementation not shown>
    }
}
```

3**Mark for Review**

The method `sitOnChair` belongs to another class and is supposed to allow the user to “sit” on a `Chair` if the chair is padded. Which of the following code segments could be used to replace `<implementation not shown>` so that `sitOnChair` will work as intended?

- I. `Chair c = new Chair(true);`
`c.sit();`
- II. `Chair c = new Chair(true);`
- III. `Chair c = new Chair(true);`
`if (c.padded) System.out.print("You are sitting.");`

(A) II only

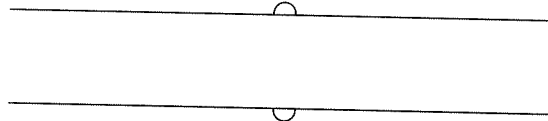
(B) III only

(C) I, II, and III

(D) None

Here's How to Crack It

Remember that an object must (1) be instantiated in the executing class, and (2) be associated only with methods that are defined in its class. Option I instantiates a `Chair` correctly, but then attempts to invoke a `sit()` method, which is not defined in `Chair` (or anywhere); eliminate (C). Option II correctly instantiates a `Chair` but does not attempt to “sit”; eliminate (A). Option III is incorrect because it also attempts to access `padded`, which is private data in `Chair` and therefore not accessible directly; eliminate (B). Since no option is valid, (D) is the answer.

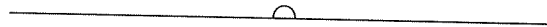
**4****Mark for Review**

Which of the following modifications, if any, would help to make the `Chair` class MOST useful to the `sitOnChair` method, based on the task attempted in Question 3?

- (A) Adding an accessor method that returns the value of `numberOfLegs`
- (B) Adding an accessor method that returns the value of `padded`
- (C) Adding a mutator method that changes the value of `numberOfLegs`
- (D) Adding a mutator method that changes the value of `padded`

Here's How to Crack It

The situation presented in Question 4 requires the information regarding whether the chair is padded. `numberOfLegs` is irrelevant here, regardless of your personal bias; eliminate (A) and (C). Since the `padded` attribute of the chair does not have to be changed, eliminate (D). The answer is (B).



Do not get “emotional” about a situation presented; follow the specifications carefully and don’t assume the programmer’s intentions.

5

 Mark for Review

Consider the following method:

```
public int halfRoot(int n)
{
    return Math.sqrt(n) / 2;
}
```

Which of the following method calls would cause a run-time error?

(A) `halfRoot(-2)`

(B) `halfRoot((int)2.0)`

(C) `halfRoot(3.0)`

(D) None will cause a run-time error.

Here's How to Crack It

The heading of the method definition requires an integer parameter. Choice (A) directly uses an integer. Even though (A) uses a negative, this doesn't result in an error. In math class, this doesn't result in a real number. However, in Java, this results in NaN. While this is not a preferred result, it is not strictly an error. Choice (B) casts a double as an integer and thus would compile and run correctly. Choice (C) gives a double parameter when an integer is required. This would cause an error. While this error would appear to cause an `IllegalArgumentException`, it will instead cause a compile-time error rather than a run-time error. The `IllegalArgumentException` would arise only if the programmer deliberately throws it, if an argument is legal in Java terms but inappropriate for the purposes of the program. (For example, a program intended to find the factorial of an integer might be designed to throw an `IllegalArgumentException` if it takes a negative parameter.) Since none of these results in a run-time error, the answer is (D).

REFERENCES

All of this data that is passed from driver classes to object classes and vice versa creates a complicated operation that's occurring behind the scenes. Consider this example: the programmer decides to write a class that represents a vinyl record player (for music), or "turntable." The driver class will then operate the turntable, turning it on and off, controlling it, and putting on/taking off the desired record.

For this program, we will have three classes: `Turntable` and `Record`, which represent each of these objects, and `PlayRecord`, which will be the driver that relates them.

In theory, we would have to perform the following steps in the driver:

- create (instantiate) a `turntable` object
- create one or more `record` objects
- place the `record` onto the `turntable`
- switch on the `turntable`

Some code from these classes might look like this:

```
public class Record
{
    // data not shown

    public Record()
    // constructor code and other methods not shown
}
public class Turntable
{
    private Record r;
    // other data not shown

    public Turntable(Record r1)
    {
        r = r1;
        // other statements not shown
    }

    // other methods not shown
}
public class PlayRecord
{
    public static void play()
    {
        Record rec = new Record();
        Turntable tt = new Turntable(rec);
        // other statements not shown
    }

    // other methods not shown
}
```

In this example, note that `Turntable` is an aggregate class, since part of its instance data involves another object. More importantly for this section, note the instantiation of the `Record` object in the driver, the “passing” of that object to the `Turntable` class, and the assignment `r = r1`. When an object is passed to another class and, as a result, received through a parameter, a **reference** is created to the object. Think of it this way: if you just picked up your favorite Taylor Swift record (you know you love her) and wanted to play it, you wouldn’t need two copies of the album. In the same way, we don’t have to create a copy of the `Record` object; we just need to create another reference to it in order for it to be used in the `Turntable` class. This is the only way the `Turntable` class can use this record,

since it was instantiated in the driver. The record can then be used in context (in this case, assigned to the data field `x`) and then the reference will disappear once the constructor is over. Since the reference is not an actual copy of the object, its life is not affected.

Another way to think of references is to think of superheroes and their secret identities. Clark Kent was just another name for the person; he was still Superman the whole time. They were the same guy with the same superpowers, the same attributes, and the same ability to perform cool superhero actions. Therefore, “Clark Kent” was simply a reference created to *refer* to the same guy.

It is extremely important to know that primitive data is not copied by reference; it is copied by value itself.

When received as a parameter, primitive data is actually copied, while object data will simply receive a new reference to the object itself. Primitives are copied by value; objects are copied by reference.

Here’s the really confusing part—or maybe not really. If a copy of a primitive data value is made, then the two copies (the original and the new one) exist exclusively. Therefore, if you change the value of one copy, the other is not affected. On the other hand, if a new reference to an object is made, then a modification of the object through the reference will be reflected by all references. Again, the superhero example: if Superman dyed his hair blond because it was all the rage, then both Clark Kent and Superman will have blond hair. They are two names that reference the same guy, so any changes will be reflected in all the references. If we suddenly decided that Superman/Clark Kent should be called “Blue Guy” instead of either of these two names, we are still referring to the same guy; he now has three *aliases* instead of two.

This is important for understanding object equality. Two objects are considered equivalent only if they are, in fact, the same object. In the superhero example above, Clark Kent is equal to Superman since they are the exact same person.

Now imagine that Superman has an identical twin brother named Duperman with alter ego Klarc Tenk. Since Duperman and Klarc Tenk are the same person, Duperman is equal to Klarc Tenk. Therefore, in Java `Duperman == Klarc Tenk` is true. However, even though they are identical twins, Superman and Duperman are different people. Therefore, Superman is not equal to Duperman. In Java, Superman and Duperman could be identified as identical twins using the `equals` method of the `Object` class. The `equals` method takes an `Object` parameter and returns `true` if and only if the two object have identical data, regardless of whether they are the same object. While the boolean statement `Superman == Duperman` is false, the statement `Superman.equals(Duperman)` is true.

this Keyword

Sometimes to call a method it may be necessary for the calling object to be able to refer to itself. When an object calls a method, an implicit reference is assigned to the calling object. The name of this implicit reference is `this`. The keyword `this` is a reference to the current calling object and may be used as an object variable. For example:

```
this.mySize
```

Use the following incomplete class declarations to answer Questions 6 and 7.

```
public class Number
{
    private int value;

    public Number(int someNum)
    {
        if (someNum >= 0)
            value = someNum;
    }

    public int changeVal(int newVal)
    {
        /* missing code */
    }

    public int getValue()
    {
        return value;
    }
}

public class NumChanger
{
    public void change()
    {
        Number n1 = new Number(5);
        Number n2 = new Number(5);
        int sum1 = n1.getValue() + n2.getValue();
        int oldn1Val = n1.changeValue(10);
        n2 = n1;
        int sum2 = n1.getValue() + n2.getValue();
        System.out.print(sum1 + " " + sum2);
    }

    //other methods not shown
}
```

6 Mark for Review

The `changeVal` method in `Number` should reassign `value` to be the value taken as the parameter and return the original value. Which of the following code segments should be used to replace `/* missing code */` so that `changeVal` will work as intended?

(A) `value = newVal;`
`return value;`

(B) `int oldVal = value`
`value = newVal;`

(C) `int oldVal = value;`
`value = newVal;`
`return value;`

(D) `int oldVal = value;`
`value = newVal;`
`return oldVal;`

Here's How to Crack It

To accomplish the task, two events must occur: `value` must be reset to the parameter value and the old value must be returned. Choice (B) does not attempt to return any value, so it cannot be correct. The remaining answer choices attempt to accomplish both events; however, if `value` is reset and its new value is returned, the second event is done incorrectly. The original value must be stored, so eliminate (A). Choice (C) stores the old value but returns the new value, since it has been reassigned to `newVal`, so it is not correct. Since (D) stores the old value, reassigns `value`, and returns the old value, (D) is the answer.



7



Mark for Review

What will be printed as a result of executing the `change` method? Assume `changeVal` in the `Number` class works as intended.

(A) 5 5

(B) 5 10

(C) 10 10

(D) None of these

Here's How to Crack It

The College Board *loves* to write multiple-choice questions with confusing object references like this. Remember that setting an object “equal” to another object merely means that the two identifiers reference the same object (like Superman and Clark Kent). `sum1` will be assigned to the sum of the values of `n1` and `n2`, which is 10, so the first number printed will be 10. Eliminate (A) and (B). `oldn1Val` is irrelevant here but `n1.changeValue(10)` makes `n1` have a value of 10. `n2 = n1` will now have variable `n2` reference the same object as `n1`, which has a value of 10 from the previous statement. Therefore, `n1.getValue` and `n2.getValue` will both return 10, so `sum2` will be $10 + 10 = 20$. As a result, the second number printed will be 20 and (C) can be eliminated. The answer is (D).

8

 Mark for Review

The following method is intended to output "Strings match." if the two strings contain identical data and otherwise print "Strings do not match."

```
public void match(String s, String t)
{
    if (/* missing code */)
        System.out.println("Strings match.");
    else
        System.out.println("Strings do not match.");
}
```

Which of the following statements could replace `/* missing code */` to allow the method to work as intended?

- I. `s.compareTo(t) == 0`
- II. `compareTo(s, t) == 0`
- III. `s == t`

(A) I only

(B) I and III only

(C) II and III only

(D) I, II, and III

Here's How to Crack It

This question is testing the proper way to determine whether two strings contain identical data. Go through each statement one at a time. Statement I uses the `compareTo` method. The `compareTo` method of the `String` class takes on another string as a parameter and returns an `int`. If the two strings contain identical data, the return is 0. (Note that this can also be applied to other objects, as the `Object` class has a similar `compareTo` method.) Therefore, Statement I will cause the method to work as intended. Eliminate (C), which does not include Statement I. Statement II also uses `compareTo`. However, since `compareTo` is a non-static method of the `String` class, it must be invoked by a `String` object. Also, the `compareTo` method takes only one parameter. Therefore, Statement II would cause a compile-time error. Eliminate (D), which contains it. Now, look at Statement III. When the `==` operator is applied to objects, the result is `true` only if the references are the same. If two objects have different references but contain identical data, the return will be `false`. The above method needs for the

return to be true in this case. Although this does not cause a compile-time or run-time error, the method will not work as intended. Therefore, eliminate (B), which contains Statement III. The answer is (A).

STATIC MODIFIER

The **static** keyword can appear in two contexts at this level. A **static variable** is an attribute that is shared among all instances of a class. When the value of this variable is changed, the alteration is reflected by all of the objects instantiated through that class. A classic example of a static variable is the high score list on a video game. Consider the video game Tetris. After you launch the game, you start a new game and play until you lose... then, since you are obsessed with Tetris, you start over. If each game that you play is considered an object of the same class, a static variable might be the high score. Every time you start (or instantiate) a new game, the high score remains the same as it was when it was initially set. Once a new high score is set, every new game will reflect the new high score, rendering the old high score non-existent (or second best, etc.).

In an object class, a static variable is declared with the rest of the instance data at the top of the class, preceded by the keyword **static**. Static variables are often used for identification numbers or for counters. Consider this short program:

```
public class Box
{
    private static int boxNumber = 0;
    // other data fields not shown

    public Box()
    {
        boxNumber++;
        // other statements not shown
    }
}

public class BoxCreator
{
    public BoxCreator()
    {
        Box box1 = new Box();
        Box box2 = new Box();
    }
}
```

As each **Box** object is instantiated in **BoxCreator**, the static variable in the **Box** class will be updated. That way, the next time a box is created, the box number value is incremented for all box objects. The static variable is not an attribute of each individual box; rather, it is a shared value among all boxes. Contrast this set-up with a non-static declaration of **boxNumber**; every time a box is instantiated, its **boxNumber** would start as 0 and then be incremented, making every box have the same **boxNumber** value of 1.

In order to show that this structure is actually working, let's create an accessor method to let the driver "see," or access, the value of `boxNumber`. This method is necessary for this result since `boxNumber` has been declared private. A non-constructor method that is designed to access and/or modify a static variable is a **static method** and must be declared as such. To add this functionality and test it, here are our new object and driver classes:

```
public class Box
{
    private static int boxNumber = 0;
    // other data fields not shown

    public Box()
    {
        boxNumber++;
        // other statements not shown
    }

    static public int getBoxNum()
    {
        return boxNumber;
    }
}

public class BoxCreator
{
    public BoxCreator()
    {
        Box box1 = new Box();
        Box box2 = new Box();
        System.out.print (Box.getBoxNum() + " boxes
        created so far.");
    }
}
```

Notice the method call from `BoxCreator` does not use an object reference variable; rather, the class is used directly. Since static variables are shared among all instances, the programmer needs to access static methods (and therefore, static variables) through the class itself.

9



Mark for Review

A class called `ComputerMouse` has a static variable `connector` and a static method `getConnector`. Which of the following statements is true based on this information?

- (A) In order to invoke `getConnector`, a new `ComputerMouse` object does not need to be instantiated; `getConnector` must be called directly through the object class.
- (B) In order to invoke `getConnector`, a new `ComputerMouse` object must be instantiated and then `getConnector` must be called through that object.
- (C) Since `connector` is declared static, `getConnector` is shared among all objects in the program.
- (D) Since `connector` is declared static, all of the methods in `ComputerMouse` must also be declared static.

Here's How to Crack It

The AP Exam tests rote knowledge as well as reasoning; this question can be answered correctly only if you know how `static` works. A static method must be called directly through the class, so (A) looks good so far, but eliminate (B). Choice (C) is incorrect because `getConnector` will be shared among all `ComputerMouse` objects, not all objects in general. Choice (D) would be correct if it said “all of the methods in `ComputerMouse` that access or mutate `connector`....” The answer is (A).

CHAPTER 7 KEY TERMS

class
source code
method
driver class
object class
header
constructor
instance variables (fields)
return type
parameters
accessor methods
mutator methods
overloading
precondition
postcondition
planning
object reference variable
aggregate class
reference
static
static variable
static method

CHAPTER 7 REVIEW DRILL

Answers to the review questions can be found in Chapter 12.

1 Mark for Review

A development team is building an online bookstore that customers can use to order books. Information about inventory and customer orders is kept in a database. Code must be developed that will store and retrieve data from the database. The development team decides to put the database code in separate classes from the rest of the program. Which of the following would be an advantage of this plan?

- I. The database access code could be reused in other applications that also need to access the database.
- II. The database access code can be tested independently. It will be possible to test the database access code before the interface is developed.
- III. A team of programmers can be assigned to work just on the code that is used to access the database. The programmers can work independently from other programmers, such as those who develop the user interface.

(A) I only

(B) III only

(C) I and II only

(D) I, II, and III

2 Mark for Review

In Java, instance fields (also known as instance variables) and methods can be designated public or private. Which of the following best characterizes the designation that should be used?

(A) Instance fields and methods should always be public. This makes it easier for client programs to access data fields and use the methods of the class.

(B) Instance fields should be either public or private, depending on whether or not it is beneficial for client programs to access them directly. All methods should be public. A private method is useless because a client program can't access it.

(C) Keep all methods and instance fields private. This enforces encapsulation.

(D) Instance fields should always be private so that clients can't directly access them. Methods can be either public or private.

3 Mark for Review

Which of the following are signs of a well-designed program?

- I. Clients know how data is stored in the class.
- II. Classes and methods can be tested independently.
- III. The implementation of a method can be changed without changing the programs that use the method.

(A) I only

(B) II only

(C) II and III

(D) I and II

4 Mark for Review

Consider the following classes:

```
public class Sample
{
    public void writeMe(Object obj)
    {
        System.out.println("object");
    }
    public void writeMe(String s)
    {
        System.out.println("string");
    }
}
```

What will be the result of executing the following?

```
Sample s = new Sample( );
String tmp = new String("hi");
s.writeMe(tmp);
```

(A) Compile-time error

(B) "object"

(C) "string"

(D) Run-time error

5 Mark for Review

Consider the following class:

```
public class Sample
{
    public void writeMe(Object obj)
    {
        System.out.println("object");
    }
    public void writeMe(String s)
    {
        System.out.println("string");
    }
}
```

What will be the result of executing the following?

```
Sample s = new Sample( );
Object tmp = new Object( );
s.writeMe(tmp);
```

☐ (A) Compile-time error☐ (B) "string"☐ (C) "object"☐ (D) Run-time error

6

 Mark for Review

Consider the following class:

```
public class Sample
{
    public void writeMe(Object obj)
    {
        System.out.println("object");
    }
    public void writeMe(String s)
    {
        System.out.println("string");
    }
}
```

What will be the result of executing the following?

```
Sample s = new Sample( );
Object tmp = new String("hi");
s.writeMe(tmp);
```

☐ (A) Compile-time error☐ (B) "object"☐ (C) "string"☐ (D) Run-time error

7 Mark for Review

Consider the following class:

```
public class Sample
{
    public void writeMe(Object obj)
    {
        System.out.println("object");
    }
    public void writeMe(String s)
    {
        System.out.println("string");
    }
}
```

What will be the result of executing the following?

```
Sample s = new Sample( );
String tmp = new Object( );
s.writeMe(tmp);
```

☐ (A) Compile-time error☐ (B) "object"☐ (C) "string"☐ (D) Run-time error

8 Mark for Review

Consider the following class:

```
public class Sample
{
    int val = 0;
}
```

Is `val` an attribute or a method?

- ☐ (A) `val` is an attribute.
- ☐ (B) `val` is a method.
- ☐ (C) `val` is both an attribute and a method.
- ☐ (D) Neither: `val` is a primitive.

9 Mark for Review

Consider the following class:

```
public class Sample
{
    public String writeMe(String s)
    {
        System.out.println("object");
    }
    public void writeMe(String s)
    {
        System.out.println("string");
    }
}
```

What will be the result of executing the following?

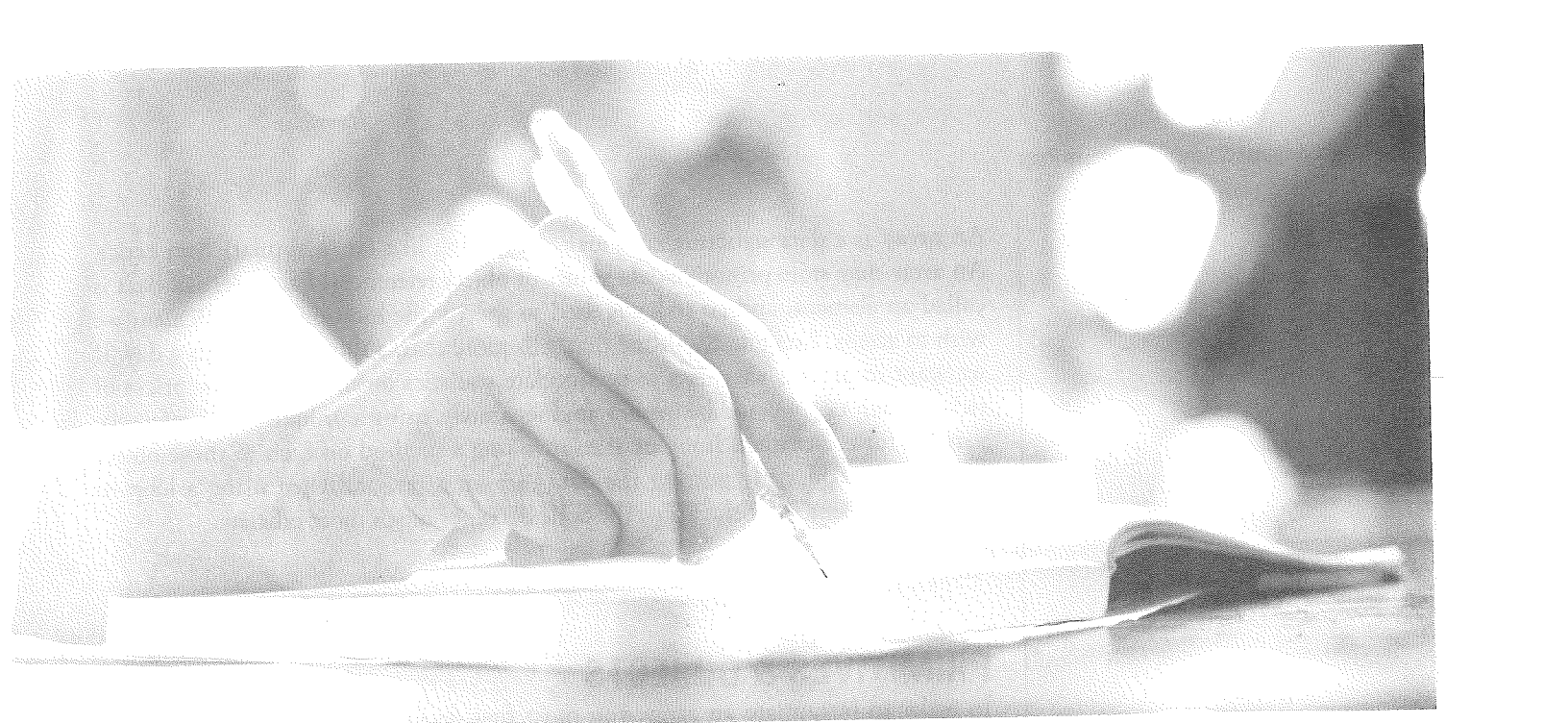
```
Sample s = new Sample( );
Object tmp = new Object( );
s.writeMe(tmp);
```

- ☐ (A) Compile-time error
- ☐ (B) "object"
- ☐ (C) "string"
- ☐ (D) Run-time error

Chapter 7 Summary

- A class is an assembly of control statements. A class can contain methods and variables. On the AP Computer Science A Exam, all class methods are public and can be accessed by other classes, and all class variables are private and cannot be accessed by other classes.
- A method is a group of code that performs a specific task.
- A method can take any number of variables of specified types as input parameters and return one value of a specified type as output. However, methods need not take any parameters or return any values.
- An object is a specific instance of a class. An object is identified by one or more references.
- For static class, all methods and variables must be called by the class name. For non-static variables, all methods and variables must be called by object references.
- On the AP Computer Science A Exam, all object variables are private and thus cannot be called by other classes, and all object methods are public and thus can be called by other classes.





Chapter 8

Array

An **array** is a data structure that makes handling like (as in similar) data easier. An array may store primitive data values or object references. Each piece of data is called an element, and an index is used to specify the location of the element we wish to access. Imagine an airline that sells more than a thousand tickets a day. If we want to process that data to accumulate statistics on origin cities or prices of tickets, each ticket could be instantiated separately, using a unique object reference variable for each ticket. Imagine then invoking a method on each of those object reference variables. By putting the data into an array, and then using a loop to access each element of the array, the code becomes much more efficient.

PRIMITIVES & OBJECTS

In order to instantiate an array, you must first decide what type of data will be contained in the array. Suppose you are creating a program that will keep track of your test grades in your APCS class. Since grades are usually represented by integer numbers, you will create an array of integers. The syntax for instantiating an array of integers is

```
int [] <identifier> = new int [<array size>];
```

This format is used if you do not yet know the values that will be stored in the array. Alternatively, if you already know the data values, you can instantiate the array using an **initializer list**:

```
int [] <identifier> = {<data1>, <data2>, ..., <data n>};
```

Notice the square brackets—these are standard symbols in Java that signify an array is present. The braces (curly brackets) indicate an initializer list. In the beginning of the semester, you do not yet know your grades. In order to create an array called `testGrades` that will eventually store your first five test grades, you could write a statement like this:

```
int [] testGrades = new int[5];
```

This array object will store your test grades and keep them organized using **index numbers** (or locations). Just as your tests might be organized as Chapter 1 Test, Chapter 2 Test, etc., your data will be accessed through the identifier of the array and index number. The tricky part is that array indices start with the number 0 instead of 1. Therefore, if your Chapter 1 Test is the first test score in the array, it will be stored at index 0. If the first test you have in APCS reviews a previous course, then it's Chapter 0 instead of Chapter 1. If you did not instantiate the array using an initializer list, then you will assign data values using the index numbers; you will do the same process to access the data regardless of how you created the array. For example, if you scored a 95 on the first test, you could write the line:

```
testGrades[0] = 95;
```

Creating an array with five scores at once might look like this,

```
int [ ] testScores = {90, 80, 100, 85, 92};
```

Let's say that, after looking over your test, you realize that your teacher made a grading error, and you actually scored a 98. You can either increment the data value or reassign it:

```
testGrades[0] += 3;
```

or

```
testGrades[0] = 98;
```

You can also perform any integer operation and also display the value to the user with that same format.

Let's step it up a notch. Suppose, after all five tests are complete, your teacher feels that the scores are too high and decides to deflate the grades. (You didn't think this book would discuss only *nice* teachers, did you?) The programmer can use a simple loop in order to **traverse** the array and change every value accordingly, rather than writing 5 separate lines. Since an array's length is well-defined, a `for` loop is usually appropriate for arrays. Provided that the array is **full**, meaning all 5 values are assigned, the following loop will traverse the array and subtract 2 points from every grade:

```
for (int index = 0; index < 5; index++)
    testGrades[index] -= 2;
```

Note the values of `index` will be 0, 1, 2, 3, and 4, even though it stores five elements. Since the index numbers start at 0, you must stop traversing the array before index 5, or you will receive an **ArrayIndexOutOfBoundsException** and the program execution will be interrupted. Free-Response Questions on the AP Exam often test your ability to write code that does not go out of bounds.

An array also has a constant value returned by `<arrayName>.length` that will return the number of elements in the array. The above loop could be written:

```
for (int index = 0; index < testgrades.length; index++)
    testGrades(index) -= 2;
```

Much better than writing multiple lines, right? The loop systematically changes every value in the array. We are just scratching the surface of the usefulness of arrays, and we have already improved our coding efficiency.

The AP Exam will require you to create, traverse, and modify arrays; free-response questions are loaded with arrays and will specifically state that an array must be created. The Course and Exam Description states, specifically, that free-response questions 3 and 4 have to do with arrays (3 is `ArrayList` and 4 is `2D Array`).

The programmer needs to ensure that every element of the array contains a meaningful value, or undesired results may occur. In an array of integers, the value of an unassigned element will default to 0. Consider the following code segment that will calculate the average of your five test grades:

```
int total = 0, len = testGrades.length;
double average;
for (int index = 0; index < len; index++)
    total += testGrades[index];
average = (double) total / len;
```

If all five of your test grades are stored in the array, this code segment will calculate the average correctly. If you did not input one of your scores, however, it will remain stored as 0 and incorrectly drag down your average, which is (definitely) an undesired result.

There is an alternative way to execute the previous code segment. This involves using an **enhanced-for loop**. An enhanced-for loop can be used to automatically go through each element of an array.

```
int total = 0, len = testGrades.length;
double average;
for (int grade : testGrades)
    total += grade;
average = (double) total / len;
```

In the enhanced-for statement, a previously declared array or an `ArrayList` (more on `ArrayLists` later) must be referenced after the colon. The loop will iterate for each element in the list, from index 0 to index `length - 1`. However, no variable is used to refer to the current index. Instead, the element located at the current index is stored as the variable declared before the colon. (The variable must be of the same type stored in the array.) Therefore, the enhanced-for loop above stores each element—one at a time in increasing order by index—of the array `testGrades` as `grade`. It then adds `grade` to `total`. Thus, the enhanced-for loop has the same functionality as the `for` loop in the previous example. The benefits of the enhanced-for loop are that it will not cause an `ArrayIndexOutOfBoundsException` when traversing an array and that it shortens the notation of the element considerably.

1



Mark for Review

Consider the following code segment:

```
final int[] a1 = {1, 2};  
int[] b1 = {3, 4};  
a1 = b1;  
System.out.print(a1[1]);
```

What is printed as a result of executing the code segment?

(A) 2

(B) 4

(C) Nothing is printed due to a compile-time error.

(D) Nothing is printed due to a run-time error.

Here's How to Crack It

This is an easy one if you know your compiler. Since `a1` is declared `final`, its reference cannot be changed. In the third line, the programmer attempts to change `a1` to reference the second array, which is not allowed. The answer is (C).

2

Mark for Review

Consider the following code segment:

```
int [] a = {1, 2, 3};  
int [] b = {4, 5, 6};  
a = b;  
for (int i = 0; i < b.length; i++)  
{  
    b[i] *= 2;  
}  
System.out.println(a[0]);
```

What is printed as a result of executing the code segment?

(A) 1

(B) 4

(C) 8

(D) Nothing is printed due to a compile-time error.

Here's How to Crack It

Remember arrays are objects. An assignment statement assigns the object reference so both `a` and `b` are pointing to the same array. If `b` is changed, `a` is changed as well. After the loop, `b = {8, 10, 12}`. Since `a` is pointing at `b`, `a[0] = 8`. The answer is (C).

3

 Mark for Review

Consider the following incomplete method:

```
public static int mod3(int[] a)
{
    int count = 0;

    // code not shown

    return count;
}
```

Method `mod3` is intended to return the number of integers in the array that are evenly divisible by 3. Which of the following code segments could be used to replace *// code not shown* so that `mod3` will work as intended?

- I.

```
for (int i = 0; i < a.length; i++)
{
    if (i % 3 == 0)
    {
        count++;
    }
}
```
- II.

```
for (int i = 0; i < a.length; i++)
{
    if (a[i] % 3 == 0)
    {
        count++;
    }
}
```
- III.

```
int i = 0;
while (a[i] % 3 == 0)
{
    count++;
    i++;
}
```

(A) I only

(B) II only

(C) I and II

(D) II and III

Here's How to Crack It

The `while` loop in option III has two problems. It will either only execute while encountering an integer in the array that is divisible by 3 (possibly stopping before traversing the entire array) or, if all the entries in the array are divisible by 3, it will go beyond the bounds of the array, causing a run-time error. This eliminates (D). Option I checks whether the index (or location) is divisible by 3. This eliminates (A) and (C). Option II checks whether the element is divisible by 3. Choice (B) is the answer.

Since objects are typically used to represent real-world phenomena, arrays are also commonly used to organize and manipulate objects. Suppose you have an object class that represents a pair of sneakers. The following code segment would instantiate an array that would store five of these objects:

```
PairOfSneakers collection[] = new PairOfSneakers[5];
```

Since each element is an object, however, the default value for each object is `null`. Since `null` is not a valid object, operations performed on `null` will result in a **NullPointerException**, another error that will halt the execution of the program. Without getting into the specifics of the `PairOfSneakers` class, a statement that could assign an instantiated pair of sneakers called `jordans` is

```
collection[0] = jordans;
```

Now, suppose `PairOfSneakers` has a `tie()` method that will tie the laces of the corresponding pair of sneakers; i.e., `jordans.tie()` will tie that pair. At this point, index 0 references the `jordans` object—remember that objects are referenced, not copied—but elements at indices 1–4 are `null`. To avoid a `NullPointerException`, check that each object is not equal to `null`.

```
for (int i = 0; i < collection.length; i++)
    if (collection[i] != null)
        collection[i].tie();
```

When using a loop to traverse an array of objects, be sure the array is full to avoid undesired results.

4



Mark for Review

Consider the following code:

```
public int[] someMethod (int[] array, int value)
{
    for (int i = 1, i < array.length - 1; i++)
        array[i - 1] += value;
    return array;
}
```

Which of the following statements is true about the method `someMethod`?

- (A) The method will return the last value in the array.
- (B) The method will cause a run-time error.
- (C) The method will not increment the first element of the array by value.
- (D) The method will not increment the last element of the array by value.

Here's How to Crack It

The return value must be an array of integers, so eliminate (A) because `array[i]` is an integer. The loop traverses the array without going out of bounds and no illegal operations are performed on the elements in the array. There will be no run-time error, so eliminate (B). The `array[0]` is incremented because the variable `i` is initialized with a starting value of 1. The method then increments `array[i - 1]`. By the same reasoning, the last element will not be incremented. The final value of `i` will be `array.length - 1`, which is normally used to find the last index within the array, but the assignment statement uses `i - 1` (which is now representing the second to the last index of the array). The loop terminates without accessing the last element, so the answer is (D).

5 Mark for Review

Consider the following code:

```
public int[] mystery (int[] array, int num)
{
    for(int i = 1, i < array.length - 1; i++)
        array[i] = 0;
    return array[0];
}
```

There is an error in method `mystery`. Which of the following modifications is needed to correct the compiler error?

- (A) Change `array.length - 1` to `array.length` in the loop statement.
- (B) Change `array[i] = 0;` to `array[i - 1] = 0;`.
- (C) Change the return statement to `return array;`.
- (D) None of these choices will make the method compile.

Here's How to Crack It

The problem with this method is the discrepancy between the return type `int[]` and the type of value that the method attempts to return, `int`; (C) addresses this issue. Choices (A) and (B), although valid, will not correct the return type issue. Choice (D) is incorrect, again, because the return type issue is keeping the method from compiling. The answer is (C).

SEARCHES

Once data is stored in an array, a common task is to methodically search the array for given data. Consider a line of parked cars in the “Curbside To Go” section of your local chain restaurant. When the server is taking the food to a particular car, she must use a given attribute—say, a license plate number—to find the correct car. Once the correct car is found, the server can deliver the food and then go back inside for the next order, beginning the process all over again...and again...and again.

This same technique is used for searching an array. In APCS, we have two methods of searching arrays, the **sequential search** and the **binary search**. Since each of these search methods uses a distinct formula to search through the arrays, they are commonly referred to as **search algorithms**.

The chain restaurant example given above is an example of a sequential search. All of the cars are “lined up” and the server begins with the first car, checking its license plate number against the one she is looking for. If it’s correct, the server delivers the food and returns. If not, she moves on to the next car and repeats the process; this all ends when either (1) the correct car is found or (2) the correct car is not in the line at all. This sequential process works just fine, although it can be very tedious. The server would have to do much more work if the desired car were at the end of the line, as opposed to at the beginning. Remember, we are programming a computer; the server cannot simply “look” at all of the plates at once, as we might in real life. The computer does not know how to “look” unless we know how to write the program that enables it!

A code segment that performs a linear search for a number target in an array of integers `nums` might look like this:

```
for (int i = 0; i < nums.length; i++)
    if (nums[i] == target)
        System.out.print("Found at " + i);
```

This segment will correctly traverse the array, searching for `target`, starting with index 0 and continuing to index `length - 1`, as desired. Remember that `nums.length` is the size of the array and therefore `nums[length]` would be out of bounds. When `target` is found, the print statement displays its index number. This process will continue until the end of the array is reached. Note that if `target` is never found, there will be no output.

When writing algorithms that traverse arrays, programmers often implement the enhanced-for loop, which is designed specifically for traversing entire arrays. In cases in which traversing the *entire* array is not the intent or would cause a runtime error, the enhanced-for loop should not be used. Assuming `nums` from the previous example would not cause such a problem, the same code segment can be written as an enhanced-for loop structure like this:

```
int loc = 0;
for (int num : nums)
{
    if (num == target)
        System.out.print("found at " + loc);
    loc++;
}
```

Each time the loop iterates, `num` will be assigned the value of the next element in the array. This structure automatically ensures that the loop does not go out of bounds. In this case, `loc` is the location or index in which the target is found.

It is important to note that the enhanced-`for` loop provides a variable that will hold each element of the array. It will NOT automatically provide the index of that element. You will have to add code to keep track of the index if you want to use it. If the variable created to hold each element is modified in any way, the array will **not** be modified.

A binary search is much more efficient; think of it as a “divide-and-conquer” mechanism. Instead of our restaurant server starting with the beginning of the line of cars, she will start with the middle car. Depending on whether the license plate is “greater” or “less” than the car she is looking for, the server will move in the appropriate direction.

There is a huge obstacle with binary searches, however, or at least for us with our current level of Java knowledge: these searches work only when there is an inherent order in the data. Suppose the top 21 students in the senior class are lined up in a row. You are looking for a student within that top 21 named Kathy, whom you have never met, and you are not sure where she is in the line. If the students are not lined up in a particular order, a sequential search is your only option, and you might need all 21 tries to find her if she is the last person in the lineup. If they are sorted in alphabetical order by first name, though, you can divide and conquer. Go to the middle person and find out her name...if her name is Sarah, you would move toward the front of the line and ignore the back. You then perform the same process, asking the name of the person halfway between the front of the line and Sarah, and then ignore the appropriate side. Within a few tries, you will have Kathy.

Here is a code example of how a binary search might look for array `nums` when searching for `target`, assuming `nums` is sorted:

```
int front = 0, back = nums.length - 1, middle = 0;
boolean isFound = false;
while((front <= back) && (!isFound))
{
    middle = (front + back) / 2;
    if (nums[middle] < target)
    {
        front = middle + 1;
    }
    else if(nums[middle] > target)
    {
        back = middle - 1;
    }
    else
    {
        isFound = true;
    }
}

if (isFound)
{
    System.out.println("Found at " + middle);
}
else
{
    System.out.println("Target Not Found");
}
```